Actividad Banco

Esta es una actividad típica que sirve como ejemplo para ilustrar el funcionamiento de la programación multihilo en todas sus variantes.

En este documento vamos a analizar cuatro variantes sobre el mismo problema, en las que iremos añadiendo funcionalidad y requisitos sobre la propuesta anterior.

Ejemplo 1: A20_Banco - Solución secuencial

> 1. Clase psp.u3.A20_Banco
> Imaginemos que 2 personas de una empresa, Luis y Manuel,van a realizar un retiro a una cuenta empresarial, lo primero que hace cada uno de ellos es comprobar el saldo, si el saldo es suficiente para el retiro, este se realiza con éxito, de lo contrario el sistema indica al usuario que no hay dinero suficiente.
> Ahora pensemos que las 2 personas lo van a realizar prácticamente al mismo tiempo.
> Van a sacar cantidades aleatorias entre 50€ y 200€. Cada uno hace 10 reintegros.
> El saldo inicial de la cuenta es de 2000€.

En este caso presentamos una solución secuencial al problema. Vamos a crear dos instancias de la clase A20_Empleado y cada una de esas instancias, una detrás de otra, irán haciendo las operaciones de reintegro de la cuenta.

No ha lugar a problemas de concurrencia, ya que por el hecho de ser un código secuencial, sólo está funcionando el hilo principal del programa que es el que se encarga de ahcer todo el trabajo sin interferencias de otros hilos.

```java
public class A20_Banco {

    public static void main(String[] args) {
        A20_Cuenta cuenta = new A20_Cuenta(2000);
        A20_Empleado emp1 = new A20_Empleado("Luís", 10, cuenta);
        A20_Empleado emp2 = new A20_Empleado("Manuel", 10, cuenta);

        // Las 10 operacions de reintegro se hacen dentro del método retirarDinero
        emp1.retirarDinero();
        emp2.retirarDinero();

        // No hace falta esperar a nada. Cuando el código llega a este punto
        // tanto Luis como Manuel han finalizado sus operaciones de reintegro
        System.out.println(emp1.getNombre() + " ha retirado " + emp1.getTotalRetirado() + "€ de la c
        System.out.println(emp2.getNombre() + " ha retirado " + emp2.getTotalRetirado() + "€ de la c
        System.out.println("En la cuenta quedan " + cuenta.getSaldo() + "€");
    }
}
```

Como se observa en la clase A20_Empleado, hay un método que se encarga de sacar el dinero de la cuenta. He optado porque las comprobaciones de saldo se hagan en el empleado (aunque son funcionalidades de la

cuenta) para diferenciar el código con el que añadiremos cuando se trabaje con hilos

```java
public class A20_Empleado {
    private String nombre;
    private int numOperaciones;
    private int totalRetirado;
    private A20_Cuenta cuenta;

    A20_Empleado(String nombre, int numOperaciones, A20_Cuenta cuenta) {
        this.nombre = nombre;
        this.numOperaciones = numOperaciones;
        this.totalRetirado = 0;

        this.cuenta = cuenta;
    }

    public void retirarDinero() {
        for (int i=0; i<numOperaciones; i++) {
            int cantidadReintegro = (int)(Math.random()*150+50);
            if (cuenta.getSaldo() > cantidadReintegro) {
                cuenta.retirar(cantidadReintegro);
                totalRetirado += cantidadReintegro;
                System.out.println(nombre + " ha realizado un reintegro de " + cantidadReintegro + "
            }
            else {
                System.out.println(nombre + " no ha podido realizar un reintegro de " + cantidadReir

            }
            System.out.println("Saldo actual en cuenta: " + cuenta.getSaldo() + "€");
        }
    }


    public int getTotalRetirado() {
        return totalRetirado;
    }

    public String getNombre() {
        return nombre;
    }

}
```

Y por último, la clase A20_Cuenta que en este ejemplo es muy sencilla, ya que no incorpora ningún tipo de funcionalidad, símplemente sirve como almacén compartido, pero no usado a la vez, de la información.

```java
public class A20_Cuenta {
    int saldo;

    A20_Cuenta(int saldo) {
        this.saldo = saldo;
    }

    public void retirar(int cantidad) {
        saldo -= cantidad;
    }

    public int getSaldo() {
        return this.saldo;
    }
}
```

Os dejo un ejemplo de la salida de este código, en el que se observa que primero Luís hace todas sus operaciones y depués Manuel hace las suyas.

```
Luís ha realizado un reintegro de 115€
Saldo actual en cuenta: 1885€
Luís ha realizado un reintegro de 187€
Saldo actual en cuenta: 1698€
Luís ha realizado un reintegro de 72€
Saldo actual en cuenta: 1626€
Luís ha realizado un reintegro de 135€
Saldo actual en cuenta: 1491€
Luís ha realizado un reintegro de 115€
Saldo actual en cuenta: 1376€
Luís ha realizado un reintegro de 147€
Saldo actual en cuenta: 1229€
Luís ha realizado un reintegro de 147€
Saldo actual en cuenta: 1082€
Luís ha realizado un reintegro de 160€
Saldo actual en cuenta: 922€
Luís ha realizado un reintegro de 83€
Saldo actual en cuenta: 839€
Luís ha realizado un reintegro de 163€
Saldo actual en cuenta: 676€
Manuel ha realizado un reintegro de 129€
Saldo actual en cuenta: 547€
Manuel ha realizado un reintegro de 168€
Saldo actual en cuenta: 379€
Manuel ha realizado un reintegro de 69€
Saldo actual en cuenta: 310€
Manuel ha realizado un reintegro de 105€
Saldo actual en cuenta: 205€
Manuel ha realizado un reintegro de 77€
Saldo actual en cuenta: 128€
Manuel ha realizado un reintegro de 62€
Saldo actual en cuenta: 66€
Manuel no ha podido realizar un reintegro de 115€
Saldo actual en cuenta: 66€
Manuel no ha podido realizar un reintegro de 69€
Saldo actual en cuenta: 66€
Manuel no ha podido realizar un reintegro de 96€
Saldo actual en cuenta: 66€
Manuel no ha podido realizar un reintegro de 85€
Saldo actual en cuenta: 66€
Luís ha retirado 1324€ de la cuenta
Manuel ha retirado 610€ de la cuenta
En la cuenta quedan 66€
```

Ejemplo 2: A20_Banco - Solución multihilo

> 1. Clase psp.u3.A20_Banco
> Imaginemos que 2 personas de una empresa, Luis y Manuel,van a realizar un retiro a una cuenta empresarial, lo primero que hace cada uno de ellos es comprobar el saldo, si el saldo es suficiente para el retiro, este se realiza con éxito, de lo contrario el sistema indica al usuario que no hay dinero suficiente.
> Ahora pensemos que las 2 personas lo van a realizar prácticamente al mismo tiempo.
> Van a sacar cantidades aleatorias entre 50€ y 200€. Cada uno hace 10 reintegros.
> El saldo inicial de la cuenta es de 2000€.

Ahora ya sí vamos a implementar la solución tal y como nos pide el enunciado, esto es, que los dos empleados intenten hacer todas las operaciones a la vez.

Por ahora, sólo tenemos que controlar aquellas partes de código que puedan ser problemáticas si varios hilos intentan ejecutarlas a la vez.

Vamos a ver un ejemplo típico de la clase principal en este tipo de problemas, donde seguiremos la siguiente secuencia:

- Creación del objeto(s) compartido
- creación de los hilos, pasándoles al menos el objeto compartido
- Ejecución de los hilos
- Espera de los hilos lanzados *(esto es opcional y dependerá del problema)*
- Muestra de resultados

La comprobación final nos sirve como aviso de que no estamos sincronizando bien los hilos y hay algo que no cuadra

```java
    public static void main(String[] args) {
        final A20_Cuenta cuenta = new A20_Cuenta(SALDO_INICIAL);
        A20_Empleado emp1 = new A20_Empleado("Luís", 100, cuenta);
        A20_Empleado emp2 = new A20_Empleado("Manuel", 100, cuenta);

        // Es lo único que cambia respecto al secuencial
        Thread t1 = new Thread(emp1);
        Thread t2 = new Thread(emp2);
        t1.start();
        t2.start();

        try {
            // Debemos esperar a que los hilos acaben para mostrar los resultados finales
            t1.join();
            t2.join();
        } catch (InterruptedException ex) { }


        System.out.println(emp1.getNombre() + " ha retirado " + emp1.getTotalRetirado() + "€ de la c
        System.out.println(emp2.getNombre() + " ha retirado " + emp2.getTotalRetirado() + "€ de la c
        System.out.println("En la cuenta quedan " + cuenta.getSaldo() + "€");

        if (emp1.getTotalRetirado()+emp2.getTotalRetirado()+cuenta.getSaldo() != SALDO_INICIAL)
            System.out.println("No cuandra el balance de la cuenta");
    }

}
```

En este caso, la clase A20_Empleado he decidido que implemente la interfaz Runnable.

En nombre lo mantengo como una propiedad de la clase, aunque cuando se empieza a ejecutar el método run también se lo pongo al Thread que está ejecutando el código.

Esto es importnte porque después, desde dentro de los métodos de cuenta no tengo acceso al objeto Runnable, pero sí al Thread que está ejecutando los métodos.

Si comparamos este código con el de la clase A20_Empleado de la solución secuencial, lo único que hemos cambiado ha sido el nombre del método `retirarDinero` por el método `run` que estamos obligados a sobreescribir por implementar la interfaz Runnable.

```java
public class A20_Empleado implements Runnable {
    private String nombre;
    private int numOperaciones;
    private int totalRetirado;
    private A20_Cuenta cuenta;

    A20_Empleado(String nombre, int numOperaciones, A20_Cuenta cuenta) {
        this.nombre = nombre;
        this.numOperaciones = numOperaciones;
        this.totalRetirado = 0;

        this.cuenta = cuenta;
    }

    @Override
    public void run() {
        for (int i=0; i<numOperaciones; i++) {
            int cantidadReintegro = (int)(Math.random()*150+50);
            if (cuenta.getSaldo() > cantidadReintegro) {
                cuenta.retirar(cantidadReintegro);
                totalRetirado += cantidadReintegro;
                System.out.println(nombre + " ha realizado un reintegro de " + cantidadReintegro + "
            }
            else {
                System.out.println(nombre + " no ha podido realizar un reintegro de " + cantidadReir

            }
            System.out.println("Saldo actual en cuenta: " + cuenta.getSaldo() + "€");
        }
    }


    public int getTotalRetirado() {
        return totalRetirado;
    }

    public String getNombre() {
        return nombre;
    }

}
```

Y por último, el gran cambio respecto a la versión secuencial, además de que estamos trabajando con hilos, es el uso del `monitor` del objeto cuenta a través de la palabra reservada `synchronized`.

Aunque ahora mismo sólo tenemos un único método que actualiza ls variable saldo, debemos evitar los problemas de acceso concurrente a ese método desde varios hilos de forma simultánea, lo que podría causar una condición de carrera y obtener resultados inconsistentes.

```java
public class A20_Cuenta {
    int saldo;

    A20_Cuenta(int saldo) {
        this.saldo = saldo;
    }

    public synchronized void retirar(int cantidad) {
        saldo -= cantidad;
    }

    public int getSaldo() {
        return this.saldo;
    }
}
```

Punto de error en el código

Aunque ya lo he advertido en la descripción del código, hay un punto débil en esta solución, ya que estamos comprobando el valor de la variable `saldo` desde el método run.
El acceso y consulta del valor de la propiedad de la cuenta desde un código no sincronizado puede ocasionar errores y usar un valor incorrecto de saldo.

En este punto, la comprobación debería pasar dentro del método retirar que sería en encargado de decidir cuándo se puede y no se puede hacer el reintegro.

Lo corregiremos en el siguiente ejemplo.

Y aquí os dejo una posible salida del código

```
Luís ha realizado un reintegro de 180€
Manuel ha realizado un reintegro de 157€
Saldo actual en cuenta: 1663€
Saldo actual en cuenta: 1663€
Manuel ha realizado un reintegro de 59€
Luís ha realizado un reintegro de 88€
Saldo actual en cuenta: 1516€
Saldo actual en cuenta: 1516€
Manuel ha realizado un reintegro de 64€
Luís ha realizado un reintegro de 65€
Saldo actual en cuenta: 1387€
Saldo actual en cuenta: 1387€
Manuel ha realizado un reintegro de 192€
Luís ha realizado un reintegro de 173€
Saldo actual en cuenta: 1022€
Saldo actual en cuenta: 1022€
Manuel ha realizado un reintegro de 136€
Luís ha realizado un reintegro de 67€
Saldo actual en cuenta: 819€
Saldo actual en cuenta: 819€
Manuel ha realizado un reintegro de 89€
Luís ha realizado un reintegro de 99€
Saldo actual en cuenta: 631€
Saldo actual en cuenta: 631€
Manuel ha realizado un reintegro de 75€
Saldo actual en cuenta: 396€
Luís ha realizado un reintegro de 160€
Manuel ha realizado un reintegro de 92€
Saldo actual en cuenta: 304€
Saldo actual en cuenta: 304€
Luís ha realizado un reintegro de 138€
Manuel ha realizado un reintegro de 153€
Saldo actual en cuenta: 13€
Saldo actual en cuenta: 13€
Manuel no ha podido realizar un reintegro de 53€
Saldo actual en cuenta: 13€
Luís no ha podido realizar un reintegro de 162€
Saldo actual en cuenta: 13€
Luís no ha podido realizar un reintegro de 125€
Saldo actual en cuenta: 13€
Luís ha retirado 970€ de la cuenta
Manuel ha retirado 1017€ de la cuenta
En la cuenta quedan 13€
```

Ejemplo 3: A21_Banco - Multihilo con esperas y avisos

> 2. Clase psp.u3.A21_Banco
> Sobre el ejemplo anterior, ahora vamos a hacer que los empleados también puedan ingresar dinero en la cuenta.
> Van a sacar cantidades aleatorias entre 50€ y 200€. Cada uno hace X reintegros.
> Van a hacer ingresos de entre 20€y 100€. Cada uno hace Y ingresos.
> El orden de las operaciones (ingresos / reintegros) debe ser aleatorio.
> El saldo inicial de la cuenta es de 2000€.
> Cada hilo, en el caso de que no pueda sacar dinero, se quedará esperando a que haya dinero suficiente. Mostrará un mensaje avisando de esta circunstancia.
> Dejaremos una espera de 100ms entre operaciones.

Si nos fijamos en la clase principal, no hay cambios significativos más allá de la comprobación final de que cuadran las cuentas entre las operaciones que han hecho los empleados y el saldo que queda en la cuenta.

También se ha producido un cambio en la creación de los empleados, ya que ahora también necesitan recibir el número de operaciones de ingreso que deben realizar.

```java
public class A21_Banco {
    final static int SALDO_INICIAL = 2000;

    public static void main(String[] args) {
        final A21_Cuenta cuenta = new A21_Cuenta(SALDO_INICIAL);
        A21_Empleado emp1 = new A21_Empleado("Luís", 5, 10, cuenta);
        A21_Empleado emp2 = new A21_Empleado("Manuel", 5, 10, cuenta);

        // Es lo único que cambia respecto al secuencial
        Thread t1 = new Thread(emp1);
        Thread t2 = new Thread(emp2);
        t1.start();
        t2.start();

        try {
            // Debemos esperar a que los hilos acaben para mostrar los resultados finales
            t1.join();
            t2.join();
        } catch (InterruptedException ex) { }


        System.out.println(emp1.getNombre() + " ha movido " + emp1.getTotalRetirado() + "€ de la cue
        System.out.println(emp2.getNombre() + " ha movido " + emp2.getTotalRetirado() + "€ de la cue
        System.out.println("En la cuenta quedan " + cuenta.getSaldo() + "€");

        if (-(emp1.getTotalRetirado()+emp2.getTotalRetirado())+cuenta.getSaldo() != SALDO_INICIAL)
            System.err.println("No cuandra el balance de la cuenta");
    }
}
```

En el caso de los empleados se ha añadido la lógica para que puedan realizar, de forma aleatoria, operaciones de ingreso y reintegro en la cuenta, siempre teniendo en cuenta que no se puede sobrepasar el número de operaciones que nos han indicado.

En esencia, el método run sigue haciendo lo mismo que en el caso anterior, aunque aquí ya hemos quitado las comprobaciones de saldo y las hemos pasado dentro de los métodos de Cuenta.

Tal y como está especificado el problema, cuando un hilo termina la ejecución de los métodos retirar/ingresar, estamos seguros de que la operación se ha realizado, por eso actualizamos el totalRetirado tras la llamada a estos métodos.

Además, tras realizar cada una de las operaciones el hilo espera 100ms antes de proceder a realizar la siguiente.

```java
public class A21_Empleado implements Runnable {

    final int MIN_INGRESO = 20;
    final int MAX_INGRESO = 100;
    final int MIN_REINTEGRO = 50;
    final int MAX_REINTEGRO = 200;
    private String nombre;
    private int numIngresos;
    private int numReintegros;
    private int totalRetirado;
    private A21_Cuenta cuenta;

    A21_Empleado(String nombre, int numIngresos, int numReintegros, A21_Cuenta cuenta) {
        this.nombre = nombre;
        this.numIngresos = numIngresos;
        this.numReintegros = numReintegros;
        this.totalRetirado = 0;

        this.cuenta = cuenta;
    }

    @Override
    public void run() {
        int ingresosHechos = 0;
        int reintegrosHechos = 0;
        boolean op;

        // Asignamos nombre al hilo
        Thread.currentThread().setName(nombre);
        while (ingresosHechos < numIngresos || reintegrosHechos < numReintegros) {
            op = new Random().nextBoolean(); // true: ingreso / false: reintegro
            if (op && ingresosHechos < numIngresos) {
                int cantidadOp = (int) (Math.random() * (MAX_INGRESO - MIN_INGRESO) + MIN_INGRESO);
                ingresosHechos++;
                cuenta.ingresar(cantidadOp);
                totalRetirado += cantidadOp;
            } else if (reintegrosHechos < numReintegros) {
                int cantidadOp = (int) (Math.random() * (MAX_REINTEGRO - MIN_REINTEGRO) + 50);
                reintegrosHechos++;
                cuenta.retirar(cantidadOp);
                totalRetirado -= cantidadOp;
            }
            try {
                Thread.sleep(100L);
                // La salida de la información hay que hacerla dentro de los métodos synchronized
            } catch (InterruptedException ex) {
            }

        }
    }

    public int getTotalRetirado() {
        return totalRetirado;
    }

    public String getNombre() {
        return nombre;
    }
}
```

El código que más cambios ha sufrido en este ejemplo es de la clase Cuenta.

Cuenta ahora tiene que implementar la funcionalidad adicional de ingresar dinero por parte de los empleados, además de gestionar la espera de los que sacan dinero, en lugar de símplemente abortar la operación y mostrat un mensaje. Esto tiene varias implicaciones en lo que a concurrencia se refiere:

- Ahora tenemos otro método que accede y actualiza la información del saldo de la cuenta, por lo que lo tendremos que hacer sincronizado.
- El méotodo ingresar está cambiando el estado de la cuenta, por lo que tendrá que avisar a posibles hilos que estén esperando ese cambio

¿Por qué hacer el notifyAll()?

Hasta ahora, los dos hilos sólo sacaban dinero de la cuenta por lo que no tenía sentido esperar, ya que la cuenta nunca se iba a recuperar y esos hilos no podrían abandonar su espera nunca.

Ahora hay hilos que pueden incrementar el saldo de la cuenta. Por este motivo los que necesitan sacar dinero deben esperar e intentar en otro momento la operación.

Es por esto que los hilos que hagan un ingreso (modifican el saldo incrementándolo) deben avisar a los que pueda haber en espera.

¿Y porqué notifyAll() en vez de notify()?
En este caso, si despertamos sólo a un hilo que quiera hacer un reintegro, puede que ese hilo necesite más dinero del que hay en la cuenta en ese momento, y volvería a hacer una espera. Sin embargo, es posible que alguno de los otros hilos que están haciendo wait sí pueda sacar dinero.
Por ese motivo los avisamos a todos y el que pueda que complete su retirada de dinero y los que no, que vuelvan al estado de espera

- En el método retirar, si no se puede sacar la cantidad solicitada, dejaremos al proceso en espera. El proceso cuando vuelva a tener el control del monitor volverá a comprobar si la operación es posible y, ejecutarla o en caso contrario, volver a esperar.

```java
public class A21_Cuenta {
    int saldo;

    A21_Cuenta(int saldo) {
        this.saldo = saldo;
    }

    public synchronized void retirar(int cantidad) {
        while (saldo-cantidad < 0) {
            try {
                System.out.println(Thread.currentThread().getName() + " no puede retirar " + cantida
                wait();
                System.out.println(Thread.currentThread().getName() + " reintentando reintegro de "
            } catch (InterruptedException ex) { }
        }
        saldo -= cantidad;
        System.out.println(Thread.currentThread().getName() + " ha sacado " + cantidad + " (saldo di
    }

    public synchronized void ingresar(int cantidad) {
        saldo += cantidad;
        System.out.println(Thread.currentThread().getName() + " ha ingresado " + cantidad + " (saldo
        // Puede haber hilos en espera de poder hacer reintegros.
        // Se les avisa a todos
        // Puede haber algunos que sigan sin poder hacer el reintegro
        // Pero otros puede que sí puedan con la cantidad actual
        notifyAll();
    }

    public int getSaldo() {
        return this.saldo;
    }
}
```

wait() siempre dentro de un while

Es uno de los principales errores cuando aplicamos el modelo productor-consumidor.
Si utilizamos 'if' un hilo puede tomar el monitor y continuar con la siguiente instrucción que hay tras el if, sin saber si hay saldo suficiente en la cuenta para hacer el reintregro.
Es ese caso podríamos dejar a la cuenta en números rojos.

Si el wait está dentro de un while, que para el caso hace la misma función que el if, cuando consiga el monitor, la siguiente línea de código que ejecutará será de nuevo comprobar la condición del while, no permitiendo nunca que se haga un reintegro si no hay saldo suficiente.

Si ejecutamos el código obtendremos salidas como esta

```
Luís ha sacado 199 (saldo disponible: 1801)
Manuel ha sacado 191 (saldo disponible: 1610)
Luís ha sacado 154 (saldo disponible: 1456)
Manuel ha ingresado 86 (saldo disponible: 1542)
Manuel ha sacado 174 (saldo disponible: 1368)
Luís ha sacado 109 (saldo disponible: 1259)
Luís ha ingresado 34 (saldo disponible: 1293)
Manuel ha ingresado 86 (saldo disponible: 1379)
Manuel ha ingresado 83 (saldo disponible: 1462)
Luís ha sacado 179 (saldo disponible: 1283)
Manuel ha sacado 163 (saldo disponible: 1120)
Luís ha ingresado 97 (saldo disponible: 1217)
Manuel ha ingresado 79 (saldo disponible: 1296)
Luís ha sacado 79 (saldo disponible: 1217)
Manuel ha sacado 196 (saldo disponible: 1021)
Luís ha ingresado 98 (saldo disponible: 1119)
Luís ha sacado 78 (saldo disponible: 1041)
Manuel ha ingresado 98 (saldo disponible: 1139)
Luís ha ingresado 35 (saldo disponible: 1174)
Manuel ha sacado 103 (saldo disponible: 1071)
Manuel ha sacado 177 (saldo disponible: 894)
Luís ha ingresado 90 (saldo disponible: 984)
Luís ha sacado 191 (saldo disponible: 793)
Manuel ha sacado 50 (saldo disponible: 743)
Manuel ha sacado 169 (saldo disponible: 574)
Luís ha sacado 175 (saldo disponible: 399)
Luís ha sacado 186 (saldo disponible: 213)
Manuel ha sacado 183 (saldo disponible: 30)
Luís no puede retirar 51 (saldo disponible: 30)
Manuel no puede retirar 166 (saldo disponible: 30)
```

Cuelgue de hilos

Si observamos las dos últimas líneas de la salida, veremos que los dos hilos se han quedado colgados en una espera.
Según el número de operaciones que indiquemos, y los valores aleatorios que vayamos obteniendo, podremos encontrarnos con que los dos hilos pueden finalizar su ejecución, uno de los dos se queda colgado en alguna espera o bien, como en el ejemplo anterior, ninguno de los dos consigue acabar.

De momento la especificación no nos ha indicado que hagamos nada al respecto y son istuaciones que pueden llegar a darse.

Ejemplo 4: A22_Banco - Multihilo con modelo productor-consumidor completo

> 3. Sobre el ejemplo anterior, ahora vamos a poner límites al uso de la cuenta.
> En la cuenta nunca podremos tener más de 2500€ ni menos de 200€
> Cada hilo mostrará mensajes indicando si en algún momento no puede realizar una operación por falta / exceso de saldo en la cuenta y se quedará esperando a que la situación cambie para intentar completar la operación.

La clase principal no tiene nngún cambio respecto al ejemplo anterior, más allá de la llamada al constructor de la cuenta, que ahora necesita un límite inferior y un límite superior.

```java
public class A22_Banco {

    final static int SALDO_INICIAL = 2000;
    final static int SALDO_MAXIMO = 2500;
    final static int SALDO_MINIMO = 200
            ;
    public static void main(String[] args) {
        A22_Cuenta cuenta = new A22_Cuenta(SALDO_INICIAL, SALDO_MAXIMO, SALDO_MINIMO);
        A22_Empleado emp1 = new A22_Empleado("Luís", 5, 10, cuenta);
        A22_Empleado emp2 = new A22_Empleado("Manuel", 5, 10, cuenta);

        // Es lo único que cambia respecto al secuencial
        Thread t1 = new Thread(emp1);
        Thread t2 = new Thread(emp2);
        t1.start();
        t2.start();

        try {
            // Debemos esperar a que los hilos acaben para mostrar los resultados finales
            t1.join();              // t1.join(1000L);
            t2.join();              // t2.join(1000L);
                                    // if (t1.getState() == Thread.State.WAITING) t1.interrupt();
                                    // if (t1.getState() == Thread.State.WAITING) t2.interrupt();
        } catch (InterruptedException ex) { }


        System.out.println(emp1.getNombre() + " ha movido " + emp1.getTotalRetirado() + "€ de la cue
        System.out.println(emp2.getNombre() + " ha movido144 " + emp2.getTotalRetirado() + "€ de la
        System.out.println("En la cuenta quedan " + cuenta.getSaldo() + "€");

        if (-(emp1.getTotalRetirado()+emp2.getTotalRetirado())+cuenta.getSaldo() != SALDO_INICIAL)
            System.err.println("No cuandra el balance de la cuenta");
    }
}
```

En la clase Empleado, sólo hemos introducido un par de cambios relacionados con la parte final de la actividad, evitar los cuelgues indefinidos de los hilos. Por ese motivo, ahora recogemos el valor de los métodos ingresar y retirar, que nos avisan si la operación se ha llevado a cabo o no.

Por lo demás, todo es exactamente igual que en el ejemplo anterior.

```java
public class A22_Empleado implements Runnable {

    final int MIN_INGRESO = 20;
    final int MAX_INGRESO = 100;
    final int MIN_REINTEGRO = 50;
    final int MAX_REINTEGRO = 200;
    final long DELAY = 100L;
    private String nombre;
    private int numIngresos;
    private int numReintegros;
    private int totalRetirado;
    private A22_Cuenta cuenta;

    A22_Empleado(String nombre, int numIngresos, int numReintegros, A22_Cuenta cuenta) {
        this.nombre = nombre;
        this.numIngresos = numIngresos;
        this.numReintegros = numReintegros;
        this.totalRetirado = 0;

        this.cuenta = cuenta;
    }

    @Override
    public void run() {
        int ingresosHechos = 0;
        int reintegrosHechos = 0;
        boolean op;

        // Asignamos nombre al hilo
        Thread.currentThread().setName(nombre);
        while (ingresosHechos < numIngresos || reintegrosHechos < numReintegros) {
            op = new Random().nextBoolean(); // true: ingreso / false: reintegro
            if (op && ingresosHechos < numIngresos) {
                int cantidadOp = (int) (Math.random() * (MAX_INGRESO - MIN_INGRESO) + MIN_INGRESO);
                // Sólo actualizamos el valor si se ha completado la operación
                if (cuenta.ingresar(cantidadOp)) {
                    ingresosHechos++;
                    totalRetirado += cantidadOp;
                }
                else {
                    // Forzamos la finalización del hilo
                    // Si vuelve a intentar otra operación se volverá a quedar
                    // colgado y ya no se podrá recuperar
                    return;
                }
            } else if (reintegrosHechos < numReintegros) {
                int cantidadOp = (int) (Math.random() * (MAX_REINTEGRO - MIN_REINTEGRO) + 50);
                // Sólo actualizamos el valor si se ha completado la operación
                if (cuenta.retirar(cantidadOp)) {
                    reintegrosHechos++;
                    totalRetirado -= cantidadOp;
                }
                else {
                    // Forzamos la finalización del hilo
                    // Si vuelve a intentar otra operación se volverá a quedar
                    // colgado y ya no se podrá recuperar
                    return;
                }
            }
            try {
```

```
                Thread.sleep(DELAY);
                // La salida de la información hay que hacerla dentro de los métodos synchronized
            } catch (InterruptedException ex) {
            }

        }
    }

    public int getTotalRetirado() {
        return totalRetirado;
    }

    public String getNombre() {
        return nombre;
    }
}

```

En la clase Cuenta, de nuevo, es donde más cambios se introducen para adaptarla a los nuevos requerimientos.

Los cambios que inteoducimos en esta ocasión son prácticamente los opuestos a los que habíamos hecho antes:

- Como ahora no se puede superar un límite de saldo, los hilos que hagan ingresos también pueden quedarse en espera. Así que, cuando se hace un reintegro tendremos que avisar a los posibles hilos que estén a la espera para que puedan comprobar si pueden completar su operación.
- Como hcabamos de decir, los hilos, en el método ingresar tendrán que comprobar si pueden hacer la operación y, en caso contrario, se quedarán esperando.
- Cuando un hilo se ve interrumpido, va a salir del método ingresar/retirar devolviendo false.

```java
public class A22_Cuenta {
    private int saldo;
    private int maxSaldo;
    private int minSaldo;

    A22_Cuenta(int saldo, int maxSaldo, int minSaldo) {
        this.saldo = saldo;
        this.maxSaldo = maxSaldo;
        this.minSaldo = minSaldo;
    }

    public synchronized boolean retirar(int cantidad) {
        while (saldo-cantidad < minSaldo) {
            try {
                System.out.println(Thread.currentThread().getName() + " no puede retirar " + cantida
                wait();
                System.out.println(Thread.currentThread().getName() + " reintentando reintegro de "
            } catch (InterruptedException ex) {
                System.err.println(Thread.currentThread().getName() + " - Salida forzada de reintegr
                return false;
            }
        }
        saldo -= cantidad;
        System.out.println(Thread.currentThread().getName() + " ha sacado " + cantidad + " (saldo di

        // Puede haber hilos en espera de poder hacer ingresos.
        // Se les avisa a todos
        // Puede haber algunos que sigan sin poder hacer el ingreso
        // Pero otros puede que sí puedan con la cantidad actual
        notifyAll();
        return true;
    }

    public synchronized boolean ingresar(int cantidad) {
        while (saldo+cantidad > maxSaldo) {
            try {
                System.out.println(Thread.currentThread().getName() + " no puede ingresar " + cantic
                wait();
                System.out.println(Thread.currentThread().getName() + " reintentando ingreso de " +
            } catch (InterruptedException ex) {
                System.err.println(Thread.currentThread().getName() + " - Salida forzada de ingreso"
                return false;
            }
        }

        saldo += cantidad;
        System.out.println(Thread.currentThread().getName() + " ha ingresado " + cantidad + " (saldo
        // Puede haber hilos en espera de poder hacer reintegros.
        // Se les avisa a todos
        // Puede haber algunos que sigan sin poder hacer el reintegro
        // Pero otros puede que sí puedan con la cantidad actual
        notifyAll();
        return true;
    }

    public int getSaldo() {
        return this.saldo;
    }
}
```

El enunciado nos plantea esta última cuestión

> ** Puede que algún hilo se quede esperando indefinidamente. Idead algún mecanismo de liberación de todos los hilos que se hayan quedado bloqueados.

La resolución de este problema se puede plantear desde dos puntos de vista.

- Forzando la interrupción de los hilos y tratando la excepción Interrupted Exceptio
- Con un cambio de estado en la cuenta y cambiando las condiciones de espera de los procesos para que contemeplen el nuevo estado.

Este código está comentado en el método main

```
        try {
             // Debemos esperar a que los hilos acaben para mostrar los resultados finales
             t1.join();              // t1.join(1000L);
             t2.join();              // t2.join(1000L);
                                     // if (t1.getState() == Thread.State.WAITING) t1.interrupt();
                                     // if (t1.getState() == Thread.State.WAITING) t2.interrupt();
                                     // ó
                                     // cuenta.setFinaliza(true);
         } catch (InterruptedException ex) { }
```

En vez de hacer un join durante un tiempo indefinido, join tiene la posibilidad de especificar cuánto tiempo queremos esperar a ese hilo.
En el código comentado hacemos que espere tiempo suficiente para asegurar que los dos hilos deberían haber acabado.

Una vez salimos de esa espera, podemos preguntar por el estado de los hilos y, en caso de que todavía estén en espera, forzar su interrupción. De ahí que en los métodos ingresar/ retirar, cuando capturamos la excepción, indiquemos devolviendo false que esa operación no se ha completado, para que no sea tenida en cuenta en el cómputo de lso movimientos de los empleados.

Aquí os dejo una salida de ejemplo que, al igual que en el caso anterior, puede acabar sin problemas, con un hilo bloqueado o con los dos hilos bloqueados.

La diferencia en este caso es que tras la espera del join, los hilos se delbloquean de manera forzada y el programa siempre acaba.

```
Luís ha ingresado 76 (saldo disponible: 2076)
Manuel ha ingresado 83 (saldo disponible: 2159)
Luís ha ingresado 25 (saldo disponible: 2184)
Manuel ha sacado 124 (saldo disponible: 2060)
Luís ha sacado 157 (saldo disponible: 1903)
Manuel ha ingresado 23 (saldo disponible: 1926)
Luís ha ingresado 34 (saldo disponible: 1960)
Manuel ha ingresado 84 (saldo disponible: 2044)
Luís ha ingresado 84 (saldo disponible: 2128)
Manuel ha ingresado 45 (saldo disponible: 2173)
Manuel ha ingresado 28 (saldo disponible: 2201)
Luís ha ingresado 22 (saldo disponible: 2223)
Luís ha sacado 77 (saldo disponible: 2146)
Manuel ha sacado 64 (saldo disponible: 2082)
Manuel ha sacado 159 (saldo disponible: 1923)
Luís ha sacado 111 (saldo disponible: 1812)
Luís ha sacado 98 (saldo disponible: 1714)
Manuel ha sacado 137 (saldo disponible: 1577)
Manuel ha sacado 131 (saldo disponible: 1446)
Luís ha sacado 104 (saldo disponible: 1342)
Manuel ha sacado 158 (saldo disponible: 1184)
Luís ha sacado 199 (saldo disponible: 985)
Luís ha sacado 130 (saldo disponible: 855)
Manuel ha sacado 118 (saldo disponible: 737)
Manuel ha sacado 169 (saldo disponible: 568)
Luís ha sacado 175 (saldo disponible: 393)
Luís ha sacado 131 (saldo disponible: 262)
Manuel no puede retirar 111 (saldo disponible: 262)
Luís no puede retirar 114 (saldo disponible: 262)
Luís - Salida forzada de reintegro
Manuel - Salida forzada de reintegro
Luís ha movido -941€ de la cuenta
Manuel ha movido -797€ de la cuenta
En la cuenta quedan 262€
```

Modelo productor-consumidor

En este último ejemplo, el código de ingresar y de retirar, junto con el de la clase Empleado y la clase principal relflejan el uso del modelo productor-consumidor.

Este modelo nos ayuda con el desarrollo de aplicaciones multihilo para que no tengamos problemas de sincronización ni de interbloqueos.